

Rapport de Projet — Gestion Pharmacie IA

DÉPARTEMENT INFORMATIQUE

Application intelligente de recommandation de médicaments : l'IA étudie les symptômes du patient et recommande les médicaments disponibles en stock.

Dépôt GitHub du projet : <https://github.com/azalira/gestion-pharmacie-assister-par-ia>

Année universitaire : 2025 - 2026 — Septembre 2026

Table des matières

1. [Résumé](#)
 2. [Membres du groupe](#)
 3. [Introduction et description du projet](#)
 4. [Objectifs](#)
 5. [Architecture générale et technologies](#)
 6. [Préparation des données \(Binôme 1\)](#)
 7. [Analyse et Intelligence Artificielle \(Binôme 2\)](#)
 8. [Back-end — API et gestion du stock \(Binôme 3\)](#)
 9. [Front-end — Interface utilisateur \(Binôme 4\)](#)
 10. [Tests et validation](#)
 11. [Résultats et démonstration](#)
 12. [Limites et perspectives](#)
 13. [Conclusion](#)
-

1. Résumé

Gestion Pharmacie IA est une application web complète d'aide à la dispensation de médicaments. Le patient décrit ses symptômes en langage naturel (ex. « *j'ai de la fièvre et mal à la tête depuis 2 jours* ») ; le moteur d'intelligence artificielle identifie les affections probables, vérifie les allergies déclarées et recommande uniquement les médicaments **réellement disponibles en stock** à la pharmacie.

Le projet couvre la chaîne complète :

- **préparation des données** (base de symptômes/médicaments, corpus BDPM),

- **intelligence artificielle** (RAG *sentence-transformers* + FAISS, et fine-tuning d'un LLM TinyLlama-1.1B en QLoRA sur les GPU gratuits de Kaggle),
- **back-end** (API REST FastAPI : authentification, CRUD médicaments, gestion du stock, ruptures),
- **front-end** (SvelteKit/Svelte 5 : espace pharmacien, espace patient, tableau de bord).

L'application a été testée de bout en bout : authentification, gestion du stock, alertes de rupture et recommandations IA fonctionnent (temps de réponse IA < 0,3 s en local).

2. Membres du groupe

Le projet a été réalisé par une équipe de **7 étudiants**, organisée en 4 binômes thématiques.

RAZAFIMAMONJY Valisoa Eli — 551

Binôme 3 — Back-end (API, gestion du stock)



FANOMEZANA Anjaratiavina Laryah — 532

Binôme 4 — Interface admin et user



ANDRIANASOLO Lahatriniaina Hery Luc — 506

Binôme 3 — Back-end (API, gestion du stock)



ANDRIAMANDROSO Harilaza Rasabotsilahy — 531

Binôme 2 — Analyse et IA (modèle, entraînement)



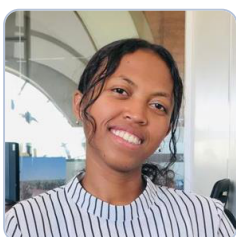
VOLOLONIRINA Larissa — 581

Binôme 1 — Préparation des données



NAMBININJANA HARY Nantenaina Fideline — 515

Binôme 1 — Préparation des données



RAVELINA MAHERY Sedra Vonjitina Joseph — 565

Binôme 4 — Interface admin et user



Répartition des responsabilités

Binôme	Membres	Rôle	Dossier
1	Larissa (581), Fideline (515)	Préparation des données	data/
2	Harilaza (531)	Analyse et IA	ml/
3	Eli (551), Hery Luc (506)	Back-end, API et stock	backend/
4	Laryah (532), Joseph (565)	Interface admin et user	frontend/

3. Introduction et description du projet

En pharmacie, le dialogue patient-pharmacien repose souvent sur l'expérience seule : face à une description de symptômes, le pharmacien doit retrouver rapidement les produits adaptés, vérifier leur disponibilité et anticiper les ruptures de stock. Le projet **Gestion Pharmacie IA** digitalise cette chaîne :

1. le **patient** saisit librement ses symptômes, son âge et ses allergies ;
2. l'**IA** analyse la demande (classification de symptômes + recherche sémantique RAG dans la base BDPM) et propose les médicaments indiqués **filtrés par le stock disponible** et les allergies ;
3. le **pharmacien** gère son stock via un tableau de bord (ajouts, ventes, seuils d'alerte, ruptures) ;
4. l'**administrateur** supervise les comptes et les données.

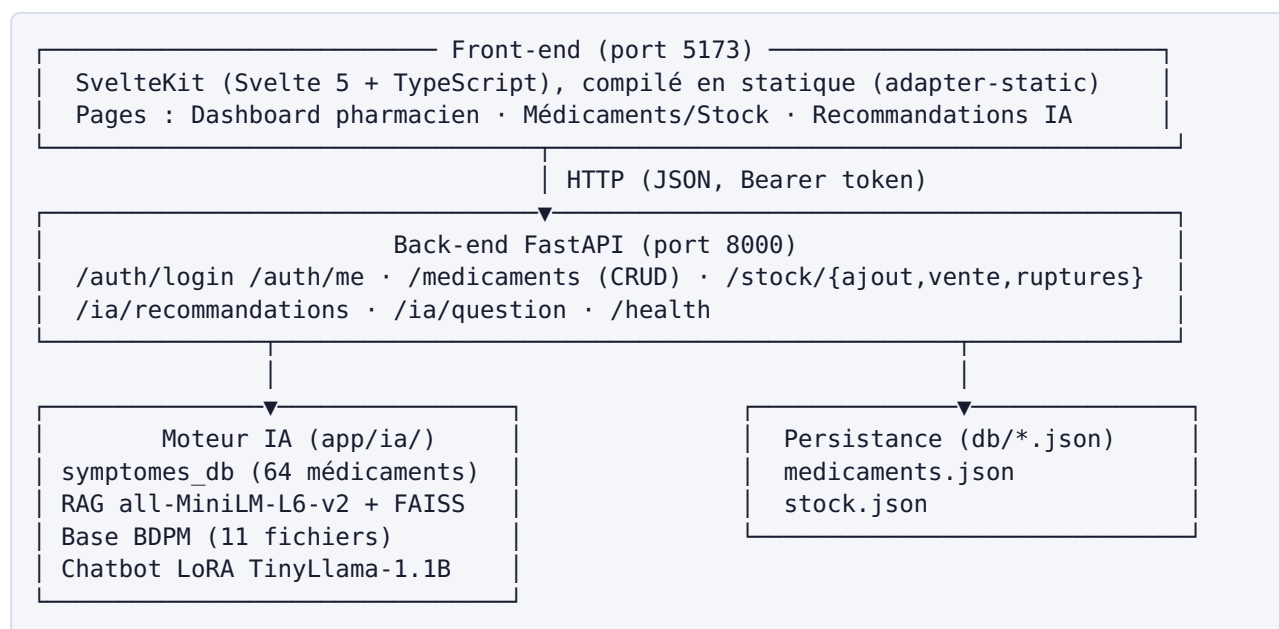
L'originalité du projet est double :

- un **moteur RAG** (Retrieval-Augmented Generation) ancré dans les données officielles BDPM (spécialités, compositions, présentations, génériques, avis SMR/ASMR, ruptures de stock) ;
- un **chatbot LLM propre au projet** : TinyLlama-1.1B fine-tuné en QLoRA sur les GPU T4 **gratuits de Kaggle**, sans GPU local — un pipeline entièrement automatisé (envoi des données → entraînement distant → récupération du modèle).

4. Objectifs

- Proposer des **recommandations de médicaments pertinentes** à partir de symptômes en français, en tenant compte de l'âge et des allergies du patient.
- **Ne recommander que ce qui est disponible** : filtrage temps réel sur l'état du stock.
- Donner au pharmacien un **suivi fin du stock** : seuils d'alerte, détection automatique des ruptures, valeur du stock.
- Construire un **chatbot pharmacien** par fine-tuning (LoRA/QLoRA) d'un petit LLM exécutable sur machine modeste, en s'affranchissant du GPU local grâce à Kaggle.
- Livrer une **application web complète et testable** : API documentée (OpenAPI), interface ergonomique, comptes de démonstration par rôle.

5. Architecture générale et technologies



Couche	Technologies
Front-end	SvelteKit, Svelte 5 (runes), TypeScript, Vite, adapter-static
Back-end	Python 3, FastAPI, Pydantic, Uvicorn
IA	sentence-transformers (all-MiniLM-L6-v2), FAISS, PyTorch, Transformers, PEFT (LoRA), bitsandbytes (QLoRA)
Données	Base BDPM (base officielle des médicaments), base symptômes interne, JSON
Entraînement distant	Kaggle Notebooks (GPU Tesla T4), API Kaggle (scripts/kaggle_gpu.py)

6. Préparation des données (Binôme 1)

- Constitution d'une **base de symptômes interne** (`symptomes_db`) couvrant **64 médicaments** courants : indications, catégories (antalgique, AINS, antibiotique, antihistaminique...), prix.
- Intégration de la **BDPM** dans `backend/app/ia/data/bdpm/` : spécialités, compositions, présentations, génériques, avis SMR/ASMR, liens de transparence, informations importantes, ruptures de stock, médicaments majeurs, gabarits de prescription.
- Nettoyage, normalisation (accents, casse) et découpage en **chunks documentaires** indexés pour le RAG (`data/vectorstore/` : `chunks.json` , `sources.json` , `index.faiss`).
- Génération du **jeu d'entraînement du chatbot** : **729 exemples** conversationnels au format ChatML (**656 entraînement / 73 validation**) dérivés de la base symptômes + base de connaissances (`ml/data/train.json` , `ml/data/val.json`).

7. Analyse et Intelligence Artificielle (Binôme 2)

7.1 Moteur de recommandation (RAG)

1. Le texte libre du patient est encodé par `all-MiniLM-L6-v2` (~90 Mo, téléchargé au premier appel).
2. Une recherche de similarité **FAISS** retrouve les maladies/médicaments les plus proches des symptômes.
3. Les maladies probables sont retournées avec leurs probabilités (ex. *Grippe/Rhume* 22 %).
4. Les médicaments sont **filtrés par les allergies** déclarées et la **disponibilité en stock**.

7.2 Chatbot fine-tuné (TinyLlama + QLoRA sur Kaggle)

Paramètre	Valeur
Modèle de base	TinyLlama-1.1B-Chat-v1.0 (Apache 2.0)
Méthode	QLoRA 4 bits — r=8, alpha=16, dropout=0.05
Jeu de données	729 exemples ChatML (656 train / 73 val)
Hyperparamètres	5 époques, batch effectif 4 (4 × accum. 2), lr 2e-4, max_length 512
Matériel	GPU Tesla T4 gratuit (Kaggle), sans GPU local
Résultat	Loss finale 2,13 — modèle LoRA 4,5 Mo, zip final 17,96 Mo

Le pipeline distant est entièrement automatisé par `scripts/kaggle_gpu.py` (`check` → `prepare` → `push` → `status --wait` → `output`) : le notebook `ml/notebooks/kaggle_train.ipynb` est exécuté sur Kaggle, puis l'adaptateur entraîné est récupéré dans `ml/models/pharma_bot_v1/` . L'inférence s'effectue en local via `ml/src/predict.py` et l'évaluation via `ml/src/evaluate.py` (métriques EM, F1, containment, latence).

8. Back-end — API et gestion du stock (Binôme 3)

API REST **FastAPI** conforme au contrat `docs/api-contrat.md` :

Méthode	Route	Rôle	Description
POST	<code>/auth/login</code>	public	Authentification, jeton Bearer
GET	<code>/auth/me</code>	connecté	Profil courant
GET/POST/ PUT/DELETE	<code>/medicaments/{id}</code>	pharmacien/ admin	CRUD complet, recherche, filtres, pagination, détection de doublons (409)
GET	<code>/stock/ruptures</code>	pharmacien/ admin	Articles en rupture (qté = 0) ou en alerte (qté ≤ seuil)
POST	<code>/stock/ajout</code>	pharmacien/ admin	Entrée de stock
POST	<code>/stock/vente</code>	pharmacien/ admin	Vente avec contrôle de stock insuffisant (409)
POST	<code>/ia/recommandations</code>	public	Symptômes + âge + allergies → maladies probables + médicaments disponibles
GET	<code>/ia/question</code>	public	Question libre via le RAG (bonus)
GET	<code>/health</code>	public	Sonde de disponibilité

- **Sécurité** : schéma Bearer, rôles `patient` / `pharmacien` / `admin`, code 401/403 selon le rôle.
- **Persistance** : fichiers JSON (`db/medicaments.json`) — simple et sans dépendance pour la démo.
- **Documentation interactive** générée automatiquement : `http://localhost:8000/docs` (OpenAPI/Swagger).

Comptes de démonstration : `pharmacien1/secret`, `patient1/secret`, `admin/admin`.

9. Front-end — Interface utilisateur (Binôme 4)

Application **SvelteKit** (**Svelte 5 en mode runes + TypeScript**), compilée en statique (`adapter-static`) et pouvant être servie directement par FastAPI.

- **Espace pharmacien** — tableau de bord : nombre de références, unités en stock, articles sous seuil/rupture, **valeur totale du stock** (en Ariary), table des alertes en temps réel.
- **Gestion des médicaments** : liste, recherche, création, modification, suppression, ajouts et ventes de stock.

- **Espace patient** : formulaire de symptômes (texte libre, âge, allergies), affichage des **maladies probables avec probabilités**, tableau des médicaments recommandés (prix, motif d'indication), avertissement médical obligatoire.
- Navigation par rôle après connexion (`pharmacien1` , `patient1` , `admin`).

10. Tests et validation

Tests manuels exécutés sur l'application complète (back-end + front-end en local) :

Vérification	Résultat
GET <code>/health</code>	200 — service opérationnel
POST <code>/auth/login</code> (<code>patient1</code> , <code>pharmacien1</code>)	jetons émis, rôles corrects
Routes protégées sans jeton	401 « Non authentifié »
GET <code>/medicaments</code> (avec jeton)	5 références retournées
GET <code>/stock/ruptures</code>	Vitamine C (rupture), Aspirine (alerte) détectées
POST <code>/ia/recommandations</code> « fièvre, maux de tête »	Grippe/Rhume 22 %, 6 médicaments proposés, 0,26 s
Filtrage allergies (pénicilline)	antibiotiques exclus des recommandations
Dashboard pharmacien (navigateur)	5 refs, 250 unités, valeur 503 500 Ar, alertes affichées
Parcours patient IA (navigateur)	recommandations cohérentes + avertissement médical

Pipeline ML évalué séparément : `ml/src/evaluate.py` (EM, F1, containment sur le jeu de validation).

11. Résultats et démonstration

- Application **fonctionnelle de bout en bout**, démarrable avec 2 commandes :

```
bash cd backend && venv/bin/uvicorn app.main:app --reload --port 8000 cd frontend && npm run dev # http://localhost:5173
```

- Recommandation IA **instantanée** (< 0,3 s) et cohérente avec le stock réel.
- Chatbot **TinyLlama-1.1B fine-tuné** (loss 2,13) entraîné à coût nul sur les GPU Kaggle — un modèle de 1,1 milliard de paramètres adapté au domaine pharmaceutique français.
- Base de connaissances **BDPM indexée** (FAISS) pour répondre aux questions libres.

12. Limites et perspectives

Limites actuelles

- Persistance JSON adaptée à la démonstration ; une base **SQLite/PostgreSQL** serait nécessaire en production (le README cible SQLite).
- Le chatbot fine-tuné n'est pas encore branché sur l'endpoint `/ia/recommandations` (le moteur RAG/classification assure le service actuel).
- 729 exemples d'entraînement restent limités pour un LLM « prêt pour la production » ; le modèle peut halluciner — **un avis médical reste indispensable**.

Perspectives

1. Brancher l'adaptateur LoRA sur le service IA (inférence CPU quantifiée).
2. Migration vers SQLite + authentification par hachage de mots de passe (bcrypt/JWT signé).
3. Enrichissement du dataset (corpus médicaux français), évaluation clinique encadrée.
4. Multi-tour conversationnel et RAG hybride (recherche lexicale + vectorielle).
5. Notifications automatiques de commande fournisseur en cas de rupture prévue (calcul CMM + délai).

13. Conclusion

Le projet **Gestion Pharmacie IA** démontre qu'une petite équipe étudiante peut livrer une application d'IA **utile et complète** : du nettoyage des données jusqu'à l'interface web, en passant par l'entraînement d'un LLM de domaine sur infrastructure gratuite. L'architecture modulaire par binômes (données → IA → API → UI) a permis un développement parallèle efficace, et l'application est aujourd'hui démontrable de bout en bout : un patient décrit ses symptômes, l'IA répond avec les médicaments réellement disponibles, et le pharmacien garde la maîtrise complète de son stock.

Rapport généré le 11 septembre 2026 — Département Informatique, promotion S6V1 & V2, 14^e promotion.